

Practico 1 - DISEÑO DE IOT Y SISTEMAS EMBEBIDOS

Autores:

- Franco Pereira
- Fabio Luzzatto

Primera Parte — Introducción al entorno ESP-IDF + VSCode

Ejercicio 1 - Creación de un proyecto

¿Qué secciones tienen los archivos generados? ¿Cuál es el propósito de cada uno?

"sample project"

- main: Contiene el código fuente principal de la aplicación
- CMakeLists.txt. Funcionan como instrucciones para que el proceso de compilación sepa qué archivos incluir y cómo construir el proyecto.

¿Qué cambios ocurren en el workspace luego de compilar? ¿Qué información provee el compilador?

- Se genera una carpeta build con el código máquina para ser ejecutado por la
- En la terminal muestra la cantidad de almacenamiento que se ocupará en la memoria flash y la cantidad de memoria RAM que se usará durante el funcionamiento del programa en la placa.

Ejercicio 2 - Parámetros de configuración (sdkconfig)

¿Qué opciones relevantes encontrás en el sdkconfig? ¿Qué diferencias aparecen entre sdkconfig y sdkconfig.old luego de guardar un cambio?

sdkconfig:

- Se agrupan los parámetros de configuración del dispositivo.
- Permite habilitar o deshabilitar funcionalidades.

Al hacer un cambio, los valores previos se respaldan automáticamente en sdkconfig.old.

Ejercicio 3 - Compilación y análisis del mapa de memoria

¿Qué información contienen esos archivos? ¿Para qué sirven en el proceso de flasheo?

Contienen metadatos para la herramienta de flasheo:

- `project_description.json`: Rutas y configuraciones del proyecto.
- `flasher_args.json`: Parámetros, tamaño y direcciones físicas de memoria donde se grabarán los binarios.

¿Qué tamaño ocupan en memoria y en qué direcciones están alojadas las variables?

Se alojan en la sección `.bss`:

- `exampleData (int)`: ocupa 4 bytes.
- `exampleArray (tamaño 12)`: ocupa 12 bytes.

¿Cómo cambia el tamaño y la dirección de `exampleArray` en el mapa de memoria al cambiar de 12 a 10?

- El tamaño reservado disminuye a 10 bytes.
- La dirección de memoria base se mantiene igual

Ejercicio 4 - Flashear y monitorear

¿Qué información muestra el proceso de flasheo en la terminal? ¿Qué significa cada sección que se transfiere (`bootloader`, `partition-table`, `app`)?

Muestra el progreso porcentual y la velocidad de transferencia.

Secciones que se transfieren:

- `bootloader`: Gestor de arranque inicial que configura el hardware para luego iniciar el software.
- `partition-table`: Mapa que divide la memoria flash en diferentes secciones.
- `app`: El firmware final compilado con nuestro código.

¿Qué imprime el programa por defecto al arrancar? ¿De dónde proviene ese mensaje?

- Imprime: Frecuencia del procesador, memoria, versión del chip.
- Proviene del proceso de inicialización de ESP-IDF y del bootloader, antes de que se ejecute la función `app_main()` de nuestro código.

Segunda Parte — "Hola Mundo" embebido: parpadeo de LED RGB

Ejercicio 1 - LED RGB WS2812C

¿Cómo funciona su protocolo de comunicación? ¿Qué representa cada bit transmitido y cuáles son los tiempos críticos?

- Se envían 24 bits para formar los colores verde, rojo, azul.
- Un "1" o "0" depende de la duración del señal, no del nivel de voltaje:
 - ◆ Bit "1": señal alto largo + bajo corto.
 - ◆ Bit "0": señal alto corto + bajo largo.
- Tiempos críticos: operan en nanosegundos (ej: 350ns a 1000ns).

Plantear en pseudocódigo cómo se controlaría el componente desde el microcontrolador.

- Configurar pin EXT_GPIO45 como salida
- Para enviar un color (24 bits):
 - ◆ Si bit es 1: Señal ALTO largo, Señal BAJO corto
 - ◆ Si bit es 0: Señal ALTO corto, Señal BAJO largo
- Mantener BAJO por más de 280 μ s para aplicar los datos (Reset).

Ejercicio 4 - Implementación del parpadeo

¿Por qué se usa el canal RMT del ESP32-S2 en lugar de generar la señal por software con un GPIO y la función de delay? ¿Qué ventajas ofrece el hardware en este caso?

- Se usa hardware porque los tiempos exigidos son de nanosegundos.
- Si se hace por software, el sistema operativo podría interrumpir la señal y usar un color o timing incorrectos.
- Ventaja: El hardware RMT maneja los tiempos precisos de forma autónoma, garantizando una señal perfecta y liberando a la CPU para otras tareas.